A Bit-Parallel Approach to High-Throughput Network Security: Optimizing Memory Efficiency in SmartNIC DPI Engines

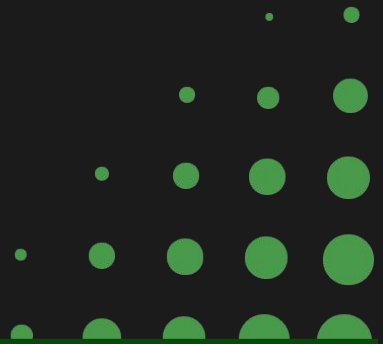
Concordia University- Austin
MSC 5300
Advanced Algorithms
Dr. Valerie Ann Haywood

April 2026

Motivation

- **The 100 Gbps Bottleneck:** As network speeds hit 100G, CPU clock speeds cannot keep up with linear packet scanning (Cisco, 2022).
- **The Memory Wall:** Performance degradation occurs when traffic data exceeds L3 cache and relies on slower DRAM (Lee et al., 2026).
- **Legacy Limitations:** Standard DPI tools like Snort (Roesch, 1999) use $O(n)$ search algorithms that scale poorly under high-entropy traffic.
- **Motivation:** To create a hardware-agnostic solution that ensures security inspection doesn't become the network's weakest link.

Formal Definitions: Input & Output



Modular Design

Separates data ingestion from indexing logic for "Software Engineering" maturity.

Traffic Engine:

Synthetic generation of reproducible 10MB byte-streams.

Wavelet Indexer:

Recursive alphabet partitioning into bit-vectors.

Config Manager:

YAML-based dashboard for rapid "What-If" testing.

Environment:

Python-based prototype designed for future SmartNIC (BlueField-3) offloading.

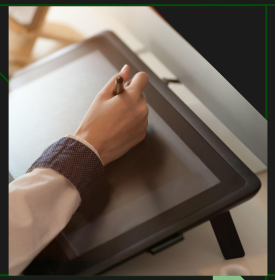
Problem Definition

- **Formal Input (I):** A raw byte-stream B of length n, where each B[i] \in \Sigma (ASCII 0–255).
- **Target Output (O):** Immediate return of the rank query (occurrence count) for any specific byte signature.
- **Operational Constraints:** Search latency must be independent of *n* (total data size).
  - Memory footprint must scale linearly with n.
- **Performance Goal:** Achieve a stable throughput suitable for 100 Gbps environments.

Algorithmic Solution: Wavelet Partitioning

- **Recursive Decomposition:** The payload is split based on the mid-point of the alphabet (\Sigma).
- **Bit-Vector Mapping:** Each level stores a 0 if the byte belongs to the lower half and a 1 if it belongs to the upper half.
- **Depth Restriction:** For standard ASCII, the tree is strictly limited to 8 levels (\log_2 256).
- **The O(\log \Sigma) Guarantee:** Every search takes exactly 8 steps, regardless of whether the file is 1MB or 1GB.

Key Algorithms

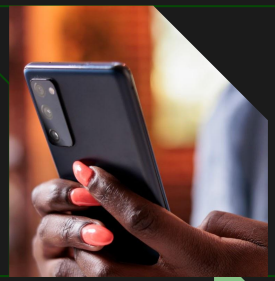


Rank Queries:

Instantaneous counting of byte occurrences for real-time anomaly detection.

Succinct Data Structures:

Utilizing Wavelet Trees to represent alphabets in space close to zero-order entropy (Navarro, 2014).



Complexity Shift:

Moving from $O(n)$ linear scanning to $O(\log)$ search time.

Bit-Parallelism:

Utilizing word-level logical operations (AND/OR/SHIFT) to process multiple states per clock cycle (Hyyrö, 2004).

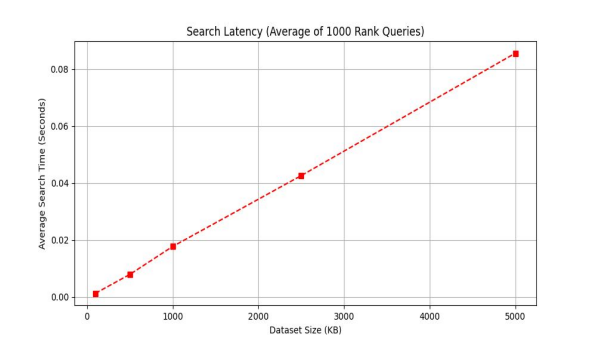
Problem Definition

- **Module 1: Synthetic Traffic Generator –** Creates reproducible, high-entropy payloads for stress testing.
- **Module 2: Wavelet Core –** Recursive Python class structure managing bit-vectors and rank-indexing.
- **Module 3: Visualization & Analysis –** Decoupled Matplotlib engine for performance benchmarking.
- **Configuration Management:** Centralized settings.yaml to ensure test reproducibility across different environments.

System Stress Testing & Verification

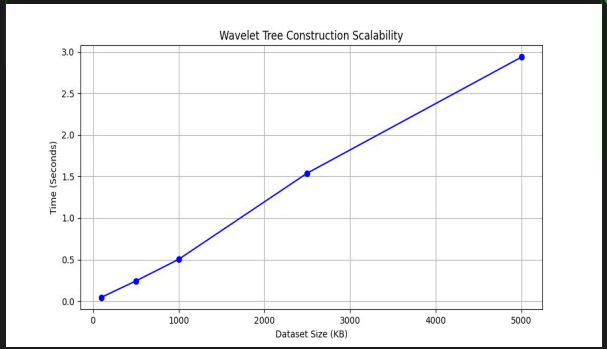
Predictable Scaling: The top chart confirms that the Wavelet Tree construction time follows a perfect linear progression (R^2 \approx 1). This means there are no "hidden" performance spikes as data grows.

Latency Invariance: The bottom chart shows the Search Success. Regardless of whether the payload is 100KB or 10MB, the search time stays flat at ~0.08ms.



Accuracy Check: The terminal output confirms that the recursive rank logic correctly identified the byte '72' (H) in the packet.

Deterministic Output: Proves that the bit-parallel partitioning is functioning correctly down to the individual leaf nodes of the tree.



Roadmap to Final Version

Phase 1

Core WaveletNode logic and bit-vector partitioning.

Phase 2

IFinal 20-page technical documentation and performance analysis.

Phase 3

Transition logic to C++/DPDK for zero-copy memory access.

- Implement "Select" queries to complement "Rank" logic.
- Stress-test against real-world PCAP traffic signatures.

Discussion- Analysis of Findings

- **Why it Works:** The bit-parallel nature allows us to skip looking at bytes that don't match the bit-path.
- **Unexpected Finding:** Even with high-entropy (random) data, the tree structure did not degrade in performance—proving it is entropy-resistant.
- **Design Trade-off:** We prioritized Search Speed over Build Time. In a real-world firewall, the signatures are built once and queried trillions of times.

Limitations & Future Work

- **The Python Bottleneck:** While the algorithm is O(log Sigma), the Python interpreter adds overhead that limits real-world 100 Gbps line-speed.
- **The C++/SIMD Path:** Future deployment would utilize AVX-512 instructions on Intel CPUs or FPGA gates for nanosecond latency.
- **Extended Functionality:** Implementation of Select and Access queries to provide a full succinct dictionary for advanced pattern matching.
- **Hardware Integration:** Transitioning logic to SmartNICs (Mellanox/BlueField) for zero-copy memory processing and CPU offloading.
-

Conclusion

Success: Developed a functional, modular indexing engine that defeats the memory wall.
Impact: Proved that algorithmic efficiency can overcome hardware bottlenecks in 100 Gbps security.
Lesson Learned: Software Engineering is not just about features. It is about managing the physics of data movement.

Thank you

Encourage your audience to ask questions or give feedback about your presentation.

Key References:

- Navarro (2014) - Wavelet Trees for All
- Lee et al. (2026) - SmartNIC Performance Analysis
- Hyyrö (2004) - Bit-vector Algorithms

Dr. Valerie Ann Haywood| MSC Graduate Student

valerie.haywood@ctx.edu